Torneo de Komikan Concurrente: informe de la Tarea 2

Sistemas Operativos, Universidad Diego Portales
Benjamín Paredes y Christián Pereira | 3 de noviembre de 2026

1. Diseño del esquema de sincronización

El torneo está escrito en C++ al estilo C, con hilos POSIX y `sem_t`. Hay un hilo `main` que hace llegar a los N jugadores de a poco, un hilo matchmaker, un hilo juez por cada mesa activa y 13 hilos de ficha por mesa (el puma y los 12 perros). Cómo compilar, ejecutar y el formato del config están en el README del repositorio. Cada recurso compartido tiene su propia primitiva:

Recurso compartido	Primitiva	Quién lo usa
Cola de jugadores (buffer circular de 1024)	`mutex_cola` y condiciones `not_full` y `not_empty`	main produce, el matchmaker consume
Arreglo `mesa_ocupada` (qué mesa está libre)	`mutex_mesas`	matchmaker y jueces
Máximo de M mesas	semáforo `mesas` (sin nombre), que parte en M	el matchmaker hace `sem_wait`, cada juez hace `sem_post`
Tablero y estado de una partida	un `pthread_mutex_t` por `Partida`	las 13 fichas y el juez de esa mesa
Aviso de Ctrl+C (`terminar`)	`atomic<int>`	el handler lo escribe, los demás lo leen
Conexiones del tablero (`ady`)	ninguna, solo se lee después de armarla	todos

Emparejamiento y límite de mesas. El matchmaker saca jugadores de la cola de a dos (el primero juega de Puma y el segundo de Perros) y hace `sem_wait` sobre `mesas`. Si las M mesas están ocupadas se bloquea ahí, sin gastar CPU (espera pasiva), y los demás jugadores siguen en la cola hasta que un juez libera una mesa con `sem_post`.

¿Cómo garantizamos la exclusión mutua en los movimientos simultáneos? Con un mutex por mesa. Cada ficha repite un ciclo: duerme un tiempo aleatorio entre `min_ms` y `max_ms`, toma el mutex de su mesa, intenta moverse, revisa si alguien ganó y suelta el mutex. Mirar qué vecinos están libres, elegir el destino y ocuparlo pasa todo dentro de la misma sección crítica, así que entre mirar y mover no se puede meter nadie, y las reglas de adyacencia y captura se validan ahí mismo. Si dos fichas quieren la misma casilla, se la queda la que toma el mutex primero y la otra la encuentra ocupada al entrar. El salto del puma cambia tres casillas (origen, perro capturado y destino) y también ocurre completo bajo el mutex, por lo que nadie ve un tablero a medias. Las fichas usan `rand_r` con semilla propia (`seed` más el jugador y el número de ficha), porque `rand()` comparte estado entre hilos.

Vida de una mesa. El juez crea el mutex, pone las fichas, lanza los 13 hilos y cada 0,5 s toma el mutex para imprimir el tablero y revisar abandono y Ctrl+C; cada movimiento y captura además sale como una línea de log. Cuando `terminada` cambia, hace `join` de las fichas, anuncia el resultado, destruye el mutex, marca la mesa como libre y recién ahí hace `sem_post`. La victoria (6 capturas o puma acorralado) la revisa la propia ficha después de cada movimiento, con el mutex tomado, porque si esperáramos al juez un perro podría alejarse y liberar al puma antes de que se notara que estaba acorralado. Por eso el juez tampoco valida cada movimiento por separado: la validación de adyacencia y captura está en el código que mueve la ficha (`mover_puma` y `mover_perro`, con `ady` y `destino_salto`), dentro de la sección crítica, para que el tablero no cambie entre validar y mover. El juez usa esa lógica al inicio (por si el config deja al puma acorralado) y se ocupa de crear, mostrar y cerrar la mesa.

Abandono. Los jugadores no son procesos reales, así que el fallo se simula con `abandona N` en el config. En su revisión periódica el juez pone `terminada` en 3 o 4 (según quién abandonó), declara ganador al oponente por abandono y sigue el cierre normal: las fichas salen solas y el mutex se destruye después del `join`, así que no queda ninguna primitiva retenida.

Ctrl+C. El handler solo escribe `terminar = 1`, una `atomic<int>` que en x86-64 es lock-free, así que es seguro hacerlo dentro de un handler. Main bloquea SIGINT con `pthread_sigmask` justo antes de crear el matchmaker y lo desbloquea justo después. Como los jueces se crean desde el matchmaker y las fichas desde los jueces, todos nacen con la señal bloqueada y solo main puede recibirla. Al recibirla, main deja de aceptar jugadores y despierta al matchmaker con `pthread_cond_broadcast`. Los jueces ven `terminar` en su revisión, ponen `terminada` en 5 y cierran sus partidas como siempre. Al final main hace `join` de los jueces que se crearon, destruye el semáforo, los mutex y las variables de condición, y libera la memoria. No usamos memoria compartida ni semáforos con nombre, así que no queda nada huérfano en el sistema.

2. Análisis de riesgos de concurrencia

Deadlock. *Escenario:* si cada casilla tuviera su propio mutex, dos fichas que quieren casillas cruzadas podrían esperarse mutuamente, y el salto del puma, que toca tres casillas, lo haría más probable. También habría deadlock si un hilo retuviera un mutex mientras espera otro recurso. *Estrategia:* un solo mutex por mesa, y ningún hilo toma dos mutex a la vez: `mutex_cola`, `mutex_mesas` y el de la mesa nunca se anidan (el juez toma `mutex_mesas` recién después de destruir el de su mesa). El `sem_wait` del matchmaker es el único bloqueo largo y lo hace sin retener ningún mutex.

El semáforo siempre se devuelve: el juez hace `sem_post` al terminar, sea por victoria, abandono o Ctrl+C, y si llega Ctrl+C mientras el matchmaker esperaba mesa, la devuelve antes de salir. A diferencia de los filósofos de `dp.c`, que necesitan dos tenedores a la vez, cada movimiento necesita un solo recurso. Para que Ctrl+C no deje hilos dormidos para siempre, las esperas de la cola también miran `terminar` y se despiertan con `broadcast`.

Livelock. *Escenario:* fichas que reintentan sin avanzar, o una partida en la que todos los hilos siguen activos pero los movimientos al azar no conducen nunca a una victoria (no hay tope de movimientos, y esa mesa se quedaría con su lugar en el semáforo). *Estrategia:* no hay busy-waiting ni `trylock`, y una ficha que no puede moverse vuelve a dormir un tiempo aleatorio, así que no hay reintentos en cadena. El puma prefiere saltar cuando puede capturar y los retardos aleatorios rompen los patrones repetidos, pero eso no garantiza que toda partida termine. En nuestras pruebas sí terminaron todas: en 60 partidas (20 mesas a la vez por cada una de tres semillas) hubo entre 12 y 260 movimientos por partida, con 56 victorias del puma y 4 de los perros. Un tope de movimientos sería una mejora posible, pero no lo agregamos porque el enunciado no dice qué pasa al alcanzarlo.

Starvation. *Escenario:* varias fichas piden a la vez el mutex de la misma mesa para ocupar las mismas posiciones, y una podría perder siempre la carrera porque el mutex de POSIX no asegura un orden. *Estrategia:* en la práctica la contención es casi nula. Cada ficha vuelve a pedir el mutex una vez cada 175 ms en promedio, o sea unos 75 intentos por segundo en toda la mesa, y cada sección crítica dura del orden de microsegundos, así que el mutex está libre casi todo el tiempo y la inanición es muy improbable, aunque POSIX no la descarta formalmente. La cola de jugadores es FIFO, y el semáforo de mesas tiene un solo hilo esperando (el matchmaker), así que los pares no compiten entre sí.

Condiciones de carrera. El estado de la partida solo se toca con el mutex de la mesa tomado. `ady` es de solo lectura, `partidas` solo la escribe el matchmaker (main la lee después del `join`) y cada mensaje sale con un solo `printf`, para que no se mezclen líneas de mesas distintas.

3. Evaluación

Comparamos cuatro formas de proteger el tablero:

Opción	Complejidad	Concurrencia	Decisión
Un mutex para todo el torneo	Muy baja	Las M mesas se serializan	Descartada
Un mutex por casilla	Alta: el salto toca 3 casillas, hay que tomar varios mutex en orden fijo y `puma_acorralado` mira todos los vecinos	La máxima dentro de una mesa	Descartada, riesgo de deadlock
Hilo coordinador por mesa con cola de movimientos	Alta: otra cola por mesa y cada ficha espera la respuesta	Igual que con un mutex, con más espera	Descartada
Un mutex por mesa	Baja	Mesas independientes, movimientos serializados dentro de cada una	Elegida

Complejidad de implementación. El código usa solo mutex, variables de condición y un semáforo, con funciones cortas y las estructuras de los ejemplos de clase (`pc.c` y `semaphore.c`), para poder explicarlo completo.

Rendimiento: throughput y latencia de movimientos. Cada sección crítica recorre a lo más 28 nodos y dura microsegundos, mientras que el retardo entre intentos de una ficha es de 50 a 300 ms (175 ms en promedio), así que el lock influye muy poco: el tiempo entre intentos lo fija el retardo aleatorio y no la sincronización. Un intento puede no terminar en movimiento (por ejemplo, un perro sin vecinos libres), por lo que el tiempo entre movimientos efectivos de una ficha puede ser mayor. Con M=1, las 10 partidas de N=20 tardaron 22,7 s en total (unos 2,3 s por partida) y sumaron 1059 movimientos (unos 106 por partida), o sea alrededor de 47 movimientos por segundo por mesa. El límite de rendimiento lo ponen los retardos que exige el enunciado y no la sincronización.

Nivel de concurrencia. Entre mesas es total: no comparten ningún mutex. Con N=20 (10 partidas) el torneo completo tardó 22,7 s con M=1, 12,2 s con M=2, 5,1 s con M=5 y 4,0 s con M=10. Con M=2 y M=5 el tiempo baja casi proporcional a M (1,9 y 4,5 veces más rápido), y con M=10 la mejora se frena (5,6 veces) porque ya lo domina la partida más larga. Dentro de una mesa los movimientos se serializan, y esa es la concurrencia que cedemos a cambio de un diseño simple y sin deadlock.

Pruebas y supuestos. El programa compila con `g++ -Wall -Wextra -std=c++17 -pthread` sin warnings y las 100 partidas de las pruebas de rendimiento (40 de los tiempos por M y 60 de las 20 mesas a la vez) terminaron con ganador. Una partida con `abandona` termina declarando ganador al oponente por abandono. Al apretar Ctrl+C a mitad de partidas, el programa termina solo con código de salida 0 y no deja procesos ni archivos nuevos en `/dev/shm`. Compilado con AddressSanitizer y UBSan (que incluye el detector de fugas), no reportó errores ni fugas ni en una partida normal (N=4, M=2) ni al interrumpirla con Ctrl+C (N=20, M=2). Supusimos que los saltos en línea recta incluyen los del triángulo (0-1-3, 0-2-7, 1-4-9, 2-6-11 y 0-5-10) y que los perros pueden moverse en cualquier dirección. La reproducibilidad con la semilla es parcial: cada ficha tiene su secuencia, pero el orden en que los hilos toman el mutex no se controla.